



An Efficient Real-Time Indoor Autonomous Navigation and Path Planning System for Drones Based on RGB-D Sensor

Ran Xiao^{1,2(✉)}, Hao Du^{1,2}, Chaowen Xu^{1,2}, and Wei Wang^{1,2,3}

¹ Institute of Applied Research Intelligent Science and Technology, Jiangsu and Chinese Academy of Sciences, Changzhou 213164, Jiangsu, China

xiaoran@arist.ac.cn

² Changzhou Research and Industrialization Center for Advanced Manufacturing Technology, Chinese Academy of Sciences, Changzhou 213164, Jiangsu, China

³ School of Automation, Nanjing University of Information Science and Technology, Nanjing 210044, Jiangsu, China

Abstract. An efficient real-time autonomous drone navigation and path planning system is proposed by using an RGB-D sensor in the GPS-denied indoor environment. Firstly, the RGB images and their corresponding depth images collected by the sensor were used for real-time pose estimation and dense point cloud map reconstruction based on ORB-SLAM (Simultaneous Localization And Mapping system based on the ORiented BRIEF feature). With the attitude angle data from the Inertial Measurement Unit (IMU) used in the pose initialization, the SLAM system can provide a more accurate pose information, and therefore a better 3D map when the drone flies with a high speed. In the second phase, a simple and efficient pathfinding program was developed. In this procedure, the point cloud map data would be translated to a map based on the octree structure, which contains the occupied and free voxels. The size of occupied nodes in the octree map is three times smaller than that of free nodes, which ensures more free area for the drone's flight. Every free voxel in the octree map was set to have enough space for the drone to cross. An improved A^* algorithm map was used to plan the route based on the octree. The simulation result showed that this efficient pathfinding method can reduce execution time and memory space significantly. In the actual flight, a local path planning system was proposed to avoid obstacles in real time.

Keywords: Indoor Visual SLAM · Drone · Octree · Collision avoidance · Path planning

1 Introduction

With the increasing of the natural and man-made disasters in recent years, the prediction and prevention of disasters have received widespread attention. At the

same time, to organize effective rescues and to reduce the losses after disasters have become very important. The small-size drones have been proved as a useful tool during missions of search and rescue [1], fire-fighting [2], target tracking [3], intelligent patrol [4], and surveillance [5]. Compared with helicopters and ground robots, drones have a lot of merits in the complex surroundings: light weight, small size and better dynamic performance and so on. Currently, for flood and earthquake disasters, small and medium-sized fixed-wing or multi-rotor drones are mostly used to carry out on site investigation; by collecting image information of the affected areas, the disaster can be quickly and accurately detected, assessed, and processed. However, it is still difficult to investigate interior situations of the GPS-denied damaged buildings. In these environments, drones should have the ability of autonomous navigation and 3D map reconstruction of surroundings (namely SLAM) and the ability of path planning.

SLAM is the key technology to improve the intelligent performance of robots, while the localization technology enables drones to know where they are, so that they can execute tasks in the unknown environment; the 3D scene reconstruction technique can not only provide necessary information for localization, navigation and obstacle avoidance, but also enable drones to perceive surroundings and identify objects. The sensors used in the current SLAM research are always lasers, cameras or multi-sensors [6–8]. Although lasers can provide the accurate range information, they are usually very expensive and heavy and not suitable for low-cost and small-size indoor drones. Cheap and light-weight cameras are a best choice for the SLAM research. Mur-Artal et al developed a convenient SLAM system for Cameras [9], but the sparse point cloud map they constructed just can provide optimization information for localization and not appropriate for the clear scene recognition and pathfinding. Lv et al [10] provided dense 3D reconstruction and the octree map based on ORB-SLAM, but they did not consider the path planning based on the 3D map. In our system, an RGB-D sensor is used in the drone (see Fig. 1) to construct both the dense point cloud map and the octree map for surveillance and pathfinding; the data of roll, pitch and yaw from the IMU provide the initial attitude information, which improves the accuracy of the pose estimation and the 3D map construction in the non-linear flight with high speed.

The popular path planning methods are mainly based the graph theory (such as Voronoi diagrams [11] and PRM [12]) and grid algorithm (e.g. A^* and Dijkstra [13] algorithm). These methods may work well in the simple 2D map, but in the complex 3D environment, they are always time-consuming and sometimes can not find an efficient path. The octree map, where the 3D space is recursively subdivided into eight octants, can compress the memory space and its resolution is adjustable according to the practical requirement. Yan et al [14] gave a PRM pathfinding method based on octree algorithm, but the computation of the connectivity evaluation in each bounding box is very complicated. Vörös [15] provides a highly efficient method for the neighbor searching by using the octree matrix instead of a voxel to represent the octree map. In the matrix octree data, each octant or node is expressed by an $n \times 3$ matrix, where n means

the octree depth level and the 3 digits in each row indicate a number in base 8 of a binary form of the corresponding level of the octree map. Based on this matrix octree representation, Rodenberg et al [16] proposed an A^* pathfinding method through a matrix octree representation of a point cloud map, they considered all the free-region neighbors including sharing faces, edges and vertexes, the disadvantage is the requirement of calculating the maximal crossing value of neighboring empty nodes.



Fig. 1. The drone equipped with the ASUS Xtion Pro-Live camera in this work.

In this research, an RGB-D sensor is used to estimate the real-time pose of the indoor drone based on ORB-SLAM, meanwhile the 3D dense point cloud map with color information is constructed for environment exploration. With the help of the pose initialization data from IMU, an accurate trajectory and a globally consistent map are achieved. In this SLAM method, IMU just provides the initial angle data in every pose estimation process, which can eliminate the effect of the IMU drift and give a precise initial angle change for the pose estimation. After completing the model construction, the saved point cloud data are compressed to the octree map represented by matrix [15] with an optimal free space resolution three times larger than the resolution of the occupied space, where the free node resolution is a little larger than the drone's maximum physical size. In the pathfinding process, the equal-size free face neighbors of the current node are considered. This method can not only give much more free space but also can make the drone pass through the free space safely. Then the A^* algorithm is used at the start and the goal at the same time for the offline path planning process, we named this method as bidirectional A^* method. The simulation results showed that this method reduced a lot of pathfinding time compared with the normal A^* algorithm. In the actual flight, a dynamic obstacle avoidance drone system is proposed, which ensures the drone to replan the path in real time when some dynamic unexpected obstacles appear in the originally planned path.

2 RGB-D SLAM System

ORB-SLAM is a precise visual-based SLAM system by using feature matching and nonlinear optimization methods [9]. It can be realized with monocular, binocular and RGB-D cameras. This SLAM system takes long time to initialize and suffers from scale drift when the monocular camera is used. For the binocular camera, the inaccuracy of the disparity calculation is a major disadvantage for the SLAM pose estimation. All these issues can be solved by using an RGB-D sensor, because this camera can provide the depth information of the feature point. In this paper, we build our indoor SLAM system by using an Xtion Pro-Live camera based on ORB-SLAM. The accurate pose estimation and dense and colorful point cloud map are constructed in real time on the on-board minicomputer in this process.

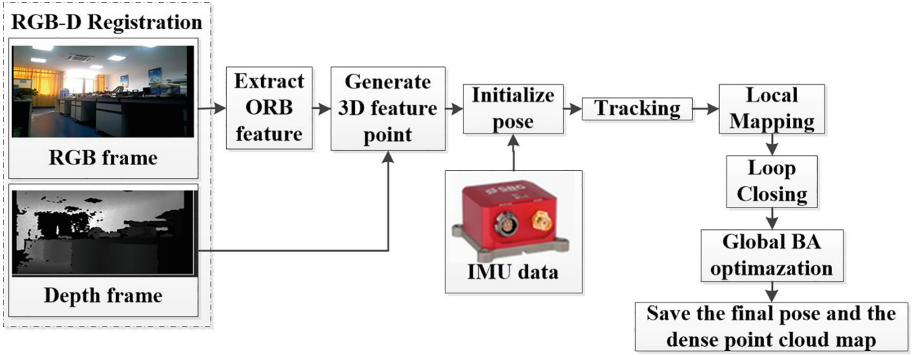


Fig. 2. Indoor SLAM work flow

In the flight experiment, the ORB-SLAM frame can work well with the only RGBD sensor when the drone flies slowly. But the 3D map is in a mess when the drone flies with a high speed (>5 m/s) especially in the non-linear flight. So an SBG IMU (Ellipse2-N) is used in the SLAM system to help improve the mapping performance, for the detailed work flow please see Fig. 2. The RGB-D camera provides both the RGB frame and depth frame (D frame) with 30 Hz, the ORB features are extracted from the RGB frame and then the corresponding depth information from the D frame is picked up to generate the 3D feature points. In the tracking period, the first step is to initialize pose with three modes: motion model, reference key frame, and relocalization. The attitude angle information from the IMU replaces the original estimated angle data as the initial pose in these three modes. The original initial pose is a transformation matrix T_{ini} which composes a rotation matrix R_{int} and a translation vector t_{int} , as shown in Eq. (1):

$$T_{ini} = \begin{bmatrix} R_{ini} & t_{ini} \\ O^T & 1 \end{bmatrix}; T_{new} = \begin{bmatrix} R_{imu} & t_{ini} \\ O^T & 1 \end{bmatrix}; \quad (1)$$

For the transformation matrix (T_{new}) in our research, the rotation matrix R_{ini} in the initial pose T_{ini} is replaced by the R_{imu} , where R_{imu} is the rotation matrix derived from the values of roll, pitch, and yaw from the IMU. Then the computation system begins with three main parallel threads: tracking, local mapping and loop closing. Finally, the global BA (Bundle Adjustment) is used to optimize the pose and the global map in the fourth thread. In this research, the color information of the points of the key frame is extracted in the tracking thread, and these colorful frames are used to construct the dense 3D colorful map by optimal transformation using their optimized pose matrix in the world coordinate system.

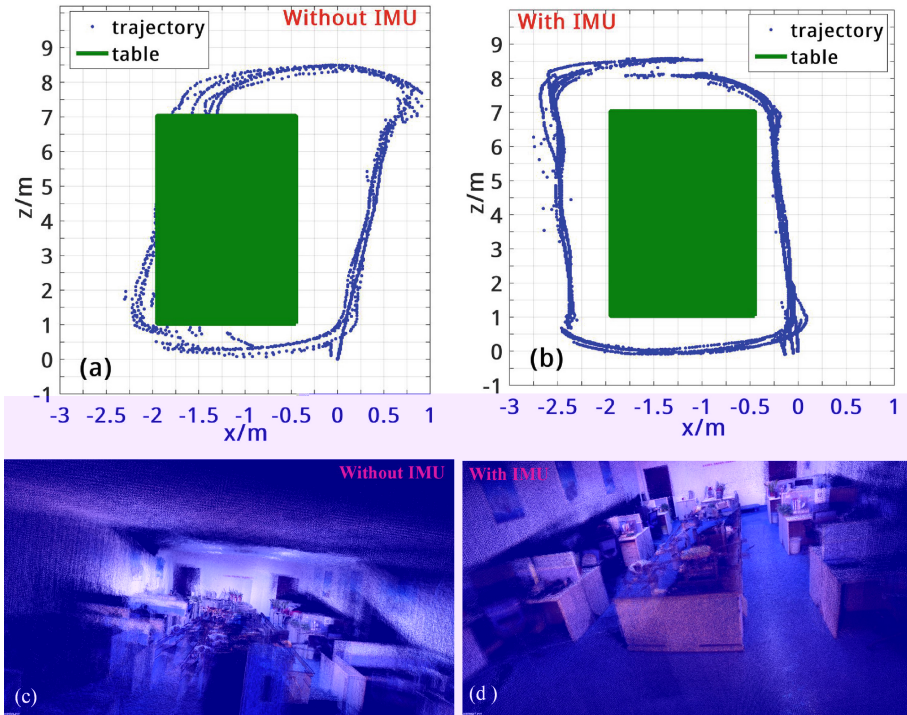


Fig. 3. The trajectories and 3D maps before and after using the IMU data. Figure (a) and (b) give the trajectories with and without IMU data, respectively. Figure (c) and (d) present the 3D dense maps with and without IMU data, respectively.

The indoor SLAM experiments are carried out with and without IMU data, respectively, and the results of the drone trajectory of 6 circles around a table and the indoor map are shown in Fig. 3, where the table is 1.5m in the width and 6m in the length. From Fig. 3(a) and (b), it is clear to see that with the help of the IMU data, the trajectory is more accurate than that without using IMU, and the pose computation rate is about 17 to 20 Hz, which can meet the

requirement of the real-time flight. For the map construction in the bottom of Fig. 3(c) and (d), it is shown that the map without using IMU data is not globally consistent, this map is too messy to be used for scene investigation and the drone navigation. Apparently the map constructed with IMU data is better for this SLAM system.

3 Pathfinding System Based on the Octree Map

3.1 Brief Introduction of the Octree Map

The octree algorithm is used to divide the 3D space to eight same-size cubes recursively till it reaches the maximum recursive depth N . Each cube or octant in its parent node can be located at one of the two sides of the split plane, which will be indicated by either a 0 (after/left/above of the split plane) or 1 (before/right/up of the split plane) in x-,y-, and z-dimension. In Fig. 4 (a), for depth level 1, the eight octants can be represented as $[0,0,0]$, $[1,0,0]$, $[0,1,0]$, $[1,1,0]$, $[0,0,1]$, $[1,0,1]$, $[0,1,1]$, $[1,1,1]$; for level 2, the eight children of node $[1,0,1]$ are $\begin{bmatrix} 1, 0, 1 \\ 0, 0, 0 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 0, 0 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 0, 1, 0 \end{bmatrix}$ and so on; for level 3, the children of node

$\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$ are $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\begin{bmatrix} 1, 0, 1 \\ 1, 1, 1 \end{bmatrix}$, $\dots$. And for level n , each octant can be represented as a matrix $\begin{bmatrix} x_1, y_1, z_1 \\ \vdots \\ x_i, y_i, z_i \\ \vdots \\ x_n, y_n, z_n \end{bmatrix}$, where x_i , y_i , and z_i represent 0

or 1 ($1 < i < n$), $[x_i, y_i, z_i]$ is the i th number of a node in level n .

The node can be encoded with only 3 binary bits for every level in the octree map. Each 3-bit number can be simplified to a single number in base 8 (from 0 to 7), where z_i and x_i are the most and least significant bits, respectively. At this point each free or occupied octant in different levels and different positions can be replaced by an integer, if the level is n , the node code contains n integers, the code of the parent node is put before the child node code, for details please see Fig. 4(b). For example, the eight octant codes of node of “5” ($n = 1$) are 50 to 57 ($n = 2$). We just need to save the digital labelled cells instead of the dense point cloud map. In this octree map, if a node has obstacle points inside, whether these points occupy the node completely or not, this node is considered as an occupied node; and the node which is not occupied by the obstacle is called the free node or empty node. It is obvious that the larger the resolution of the nodes is, the smaller the free area can be used in the flight area. The small octree map resolution can give more free region for the drone, but the drone needs enough space to pass through each node safely. So two different resolutions are used for the occupied nodes and free nodes. In our research, if the eight face neighbors of the initial free node are free, this node is considered as a new free node.

Therefore, the new free node resolution is three times larger than the occupied node. And only these larger size free nodes are used as the area for pathfinding. All the free nodes mentioned below are large size free nodes.

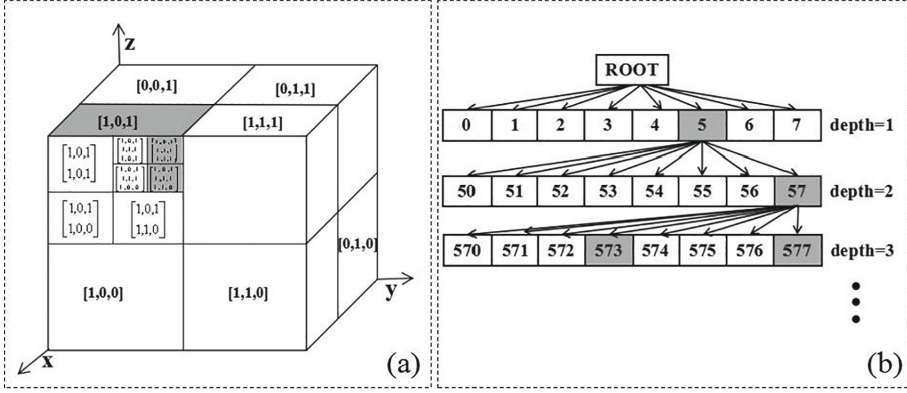


Fig. 4. Subdivision of an octree. Figure (a) and (b) give the matrix and digital code representation of the octants, respectively. White octants mean free nodes, and gray octants are occupied nodes.

3.2 Pathfinding Based on A* Algorithm

The free nodes collected in Sect. 3.1 by using the octree subdivision method are used for the drone path planning. In our research, both the global off-line and the real-time pathfinding methods are used in the drone to improve the performance of dynamic obstacle avoidance and the safety of real-time planning.

Global Bidirectional A* Pathfinding Algorithm. The free area separated out from the map are used for the A* path-finding method, which can speed up the efficiency of the path planning. The principle of the normal A* algorithm is as follows in Eq. (2):

$$f(M) = g(M) + h(M) \quad (2)$$

where $f(M)$ is the estimated cost from the start node S to the goal node G through the current node M , $g(M)$ is the practical cost from S to M , $h(M)$ is the estimated cost from M to G , in this research it computes the Manhattan distance between M and G . In the path planning, the next node with the minimum total cost $f(M)$ is chosen. According to the octree division method, the map will be processed as a big cube, this cube is normally larger than the actual map, and accordingly the collected free area is always outside of the drone's flight scope. So only these free nodes in the octree map inside the 3D map are taken into account. What's more, only the equal-size free face neighbor nodes are considered to enable the drone to pass safely and improve the pathfinding

efficiency simultaneously. In the off-line path planning phase, unlike the conventional A^* method, we apply the bidirectional A^* algorithm to search the route from the departure and destination points parallelly, the specific steps are show in Algorithm 1. In this algorithm, *OpenSet1* and *OpenSet2* are two sets to save the not evaluated nodes, *CloseSet1* and *CloseSet2* are two sets to save nodes which are already evaluated, *ComeFrom1* and *ComeFrom2* are to save the relation between the current node and its previous node with the minimum actual cost. And $g1(N1)$, $g2(N2)$ are the actual costs from S to node $N1$ and from node G to node $N2$, respectively, where $N1$ and $N2$ are the current nodes of the bidirectional A^* method from S and G ; $f1(N0)$ and $f2(N0)$ describe the total costs from S to $N2$ and from G to $N1$ through node $N0$, respectively. $\text{Distance}(N1, N0)$ computes the distance from $N1$ to $N0$, $\text{neighbors}(N1)$ gives the face neighbors of $N1$ in the scope of free nodes. By using this search method, the path data are saved in vector P by the combination of two routes from S and G , and the final global path graph P is sent to the drone's flight controller.

In order to prove the advantage of the bidirectional A^* algorithm, both the path planning performance of the bidirectional A^* and the normal A^* algorithms are computed in our research in a complicated map with the area of $100 \times 100 \times 50 \text{ m}^3$ and with the map point number of 209,361. In the simulation, we assume that the drone flies from start point (13, 92, 45) m to the goal point (97, 10, 3) m. We simulate these two pathfinding methods in different free node octree map resolutions of R , $2R$, $4R$ and $8R$ respectively, where R is 0.38 m. The computer system in this simulation has an Intel Core i7-4790 CPU (3.60 GHz) processor and 8.00 GB of RAM with Ubuntu14.04 64 bits. The simulation result in Table 1 shows that the bidirectional A^* and the normal A^* method have the almost same path distance in different resolutions. The number of the empty leaf nodes and the computation time of different octree map resolutions are shown in Fig. 5. From this figure, it is obvious that the larger the resolution is, the less the empty node number is, and the less the flight area (resolution $\times$ (empty node number)) is. The computation time also varies inversely with the octree map resolution. In order to take account of both computation time and the flight area, the free node resolution is larger than the occupied node's and the free node resolution is set as a value which is a little bigger than the drone's physical size to enable this drone to pass through safely. Comparing the computation time of the A^* and bidirectional A^* methods, we can see that the former is almost one order of magnitude larger then the latter. And the smaller the resolution is, the larger of the two kinds of computation time difference value is.

According to the above simulation analysis, the bidirectional A^* method is used for searching an efficient route in the global pathfinding. And the result of the global path planning in the real indoor environment is expressed in Fig. 6(a), where the map size is $6.71 \times 12.58 \times 3.16 \text{ m}^3$ and the size of the free node is 0.80 m. Figure 6(b) shows the octomap of the indoor environment with the occupied node resolution of about 0.27 m.

Algorithm 1. Bidirectional A^* algorithm

```

OpenSet1.push(S); OpenSet2.push(G)
CloseSet1=empty set; CloseSet2=empty set
ComeFrom1=empty map; ComeFrom2=empty map
g1[S]=0; g2[G]=0
f1[S]=Distance(S, G); f2[G]=Distance(G, S)
while OpenSet1!=Null & OpenSet2!=Null do
    N1=Node in OpenSet1 with smallest f1; OpenSet1.remove(N1) // Extract the
    node with smallest  $f1$  and remove this node from OpenSet1
    N2=Node in OpenSet2 with smallest f2; OpenSet2.remove(N2)
    if N2 in neighbors(N1) then
        return ComeFrom1, ComeFrom2, g1, g2
    end if
    for N0 in neighbors(N1) do
        gnew=g1(N1)+Distance(N1, N0) // gnew is the new actual cost
        if N0 not in CloseSet1 or gnew < g1(N0) then
            g1(N0)=gnew
            f1(N0)=g1(N0)+Distance(N2, N0)
            OpenSet1.push(N0); CloseSet2.push(N0); ComeFrom1[N0]=N1
            for N0 in neighbors(N2) do
                gnew=g2(N2)+Distance(N2, N0)
                if N0 not in CloseSet2 or gnew < g2(N0) then
                    g2(N0)=gnew
                    f2=g2(N0)+Distance(N1, N0)
                    OpenSet2.push(N0); CloseSet2.push(N0); ComeFrom2[N0]=N2
                end if
            end for
        end if
    end for
    end if
end while

```

Table 1. The path distances of different octree resolutions.

Resolution/m	R	$2R$	$4R$	$8R$
Path cost/m	207.68	208.05	208.83	213.47

Dynamic Obstacle Avoidance System Proposal. The former part of Sect. 3.2 gives the off-line global path planning. This method is adequate for the static environment, but in the actual flight, there are always some dynamic obstacles. So the local dynamic path planning method in the drone's flight is proposed as in Fig. 7. In the task of flight mission, the drone flies according to the path P from the start point S and detects the surrounding environment in real time. Then the on-board computing system judges whether there are obstacles in the area of next 1 to T nodes after the current node in the current path, where

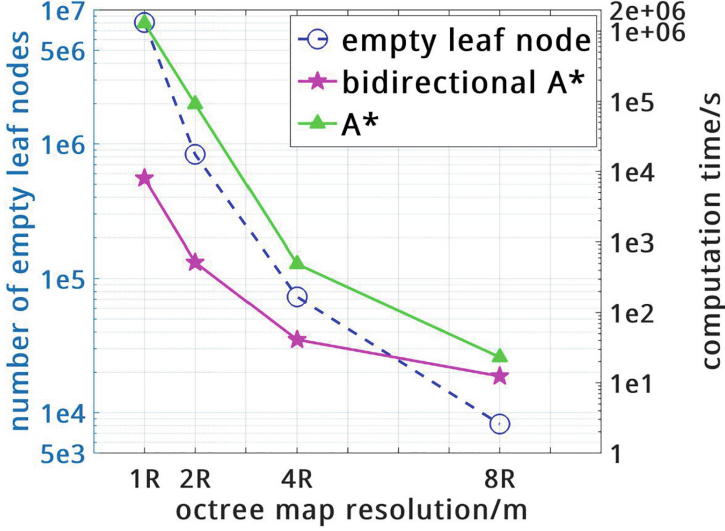


Fig. 5. The empty leaf node number and the computation time of different octree map resolutions where $R = 0.38$ m.

$T * R$ is the safe distance of the drone to obstacles and R is the size of the empty node. If there is no obstacle until the drone arrive at goal point, this real-time navigation ends successfully. If there are obstacles in the path, these occupied nodes will be set as set O . The next step is to search out whether there is a route from the current node N to the next node in the path graph P skipping the set O to the destination G by using the normal A^* method. If there is a route, the node is set as W , and the original route is repaired by replacing the path from current node N to node W by the new path, noted as new path P . And then the drone will fly along path P and detect the surrounding environment in a circular manner in real time till it arrives at point G . If there is no new path, this pathfinding process ends in failure.

4 Discussion and Summary

In this paper, we discussed the vision-based SLAM system based on ORB-SLAM and the real-time path planning system with an RGB-D camera. The SLAM process is very critical because an accurate and perfect map is a prerequisite for the path planning. The experiment results showed that the trajectory and the point cloud map were more accurate with the IMU data using in the pose initialization process. After reconstructing the colorful dense point cloud map, in order to increase the efficiency of the A^* pathfinding method, we use the octree algorithm to divide the space to the empty area and the occupied area. In the octree map, the empty node's resolution is three times larger than the occupied node's, so that more space can be used for the drone to pass through

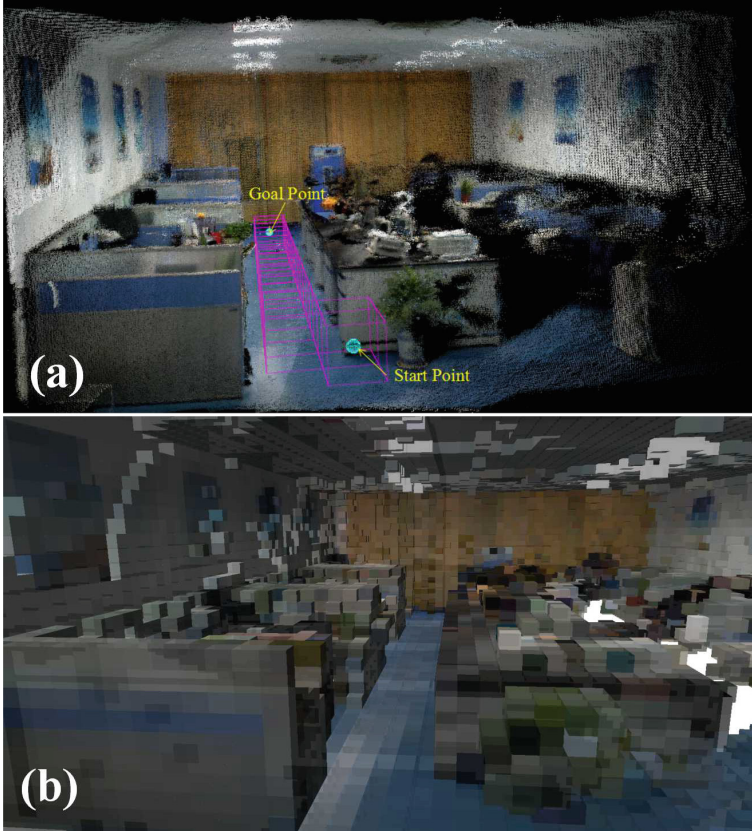


Fig. 6. The global path planning in indoor environment. In Figure (a), the magenta square area give the flight area from the start point to the goal point. Figure (b) gives the octomap of the indoor environment.

safely. Two kinds of the A^* algorithm are used in the process of pathfinding: the bidirectional A^* algorithm and normal A^* algorithm. The former method is used for the global path planning, which starts the A^* method from the start and goal points parallelly and improve the path planning efficiency compared with the normal A^* algorithm starting from one point. The latter method is used when the drone flies to avoid obstacles in real-time in the real flight operation in the local path planning. The global path-finding simulation in the complicate map with different octree resolutions shows that the bidirectional A^* method can improve the computation time significantly.

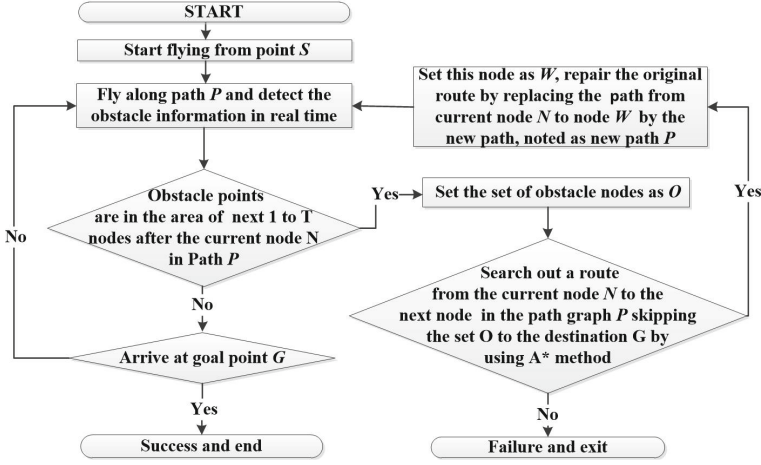


Fig. 7. The path-finding method with dynamic obstacle avoidance work flow.

Acknowledgement. This research was supported by National Key Research and Development Program of China (No. 2017YFC0806500).

References

1. Graham-Rowe D (2010) Cheap drones could replace search-and-rescue helicopters. *New Scientist* 207(2769):20
2. Qin H, Cui JQ, Li J et al (2016) Design and implementation of an unmanned aerial vehicle for autonomous firefighting missions. In: *IEEE international conference on control and automation*. IEEE, Kathmandu, pp 62–67
3. Chakrabarty A, Morris R, Bouyssounouse X et al (2016) Autonomous indoor object tracking with the Parrot AR.Drone. In: *International conference on unmanned aircraft systems*. IEEE, Arlington, pp 25–30
4. Qiu Z, Zhang K, Dong Y (2017) Research on the system of patrol unmanned aerial vehicle (UAV) docking on charging pile based on autonomous identification and tracking. In: *International conference on intelligent and interactive systems and applications*. Springer, Cham, pp 517–523
5. Geng L, Zhang YF, Wang JJ et al (2013) Mission planning of autonomous UAVs for urban surveillance with evolutionary algorithms. In: *IEEE international conference on control & automation*, Chengdu, vol 45, no 5, pp 828–833
6. Doer C, Scholz G, Trommer GF (2017) Indoor laser-based SLAM for micro aerial vehicles. In: *Gyroscopy & Navigation*, vol 8, no 3, pp 181–189 (2017)
7. Liénard J, Vogs A, Gatzliolis D, Strigul N (2016) Embedded, real-time UAV control for improved, image-based 3D scene reconstruction. *Measurement* 81:264–269
8. López E, Barea R, Gómez A, Nemra A (2016) Indoor SLAM for micro aerial vehicles using visual and laser sensor fusion. In: *Robot 2015: second Iberian robotics conference*. *Advances in intelligent systems and computing*, vol 417. Springer, Cham

9. Mur-Artal R (2017) Tardós JD (2017) ORB-SLAM2: an open-source SLAM system for monocular, stereo, and RGB-D cameras. *IEEE Trans Robot* 33(5):1255–1262
10. Lv Q, Lin H, Wang G, Wei H, Wang Y (2017) ORB-SLAM-based tracing and 3D reconstruction for robot using Kinect 2.0. In: *Control and Decision Conference, Chongqing*
11. Beard RW, McLain TW, Goodrich M, Anderson EP (2002) Coordinated target assignment and intercept for unmanned air vehicles. *IEEE Trans Robot Autom* 18(6):911–922
12. Mark H, Overmars Mark H, Overmars T (1992) A random approach to motion planning. *RUU-CS* (92-32)
13. Dijkstra EW (1959) A note on two problems in connexion with graphs. *Numerische Mathematik* 1(1):269–271
14. Yan F, Liu YS, Xiao JZ, Yan F, Liu YS, Xiao JZ (2013) Path planning in complex 3D environments using a probabilistic roadmap method. *Int J Autom Comput* 10(6):525–533
15. Vörös J (2000) A strategy for repetitive neighbor finding in octree representations. *Image Vis Comput* 18(14):1085–1091
16. Rodenberg O, Verbree E, Zlatanova S, Zlatanova S (2016) Indoor A* pathfinding through an octree representation of a point cloud. *ISPRS Ann Photogram Remote Sens Spat Inf Sci IV-2/W1*:249–255